SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE CODE: ITA 06 **COURSE NAME: MACHINE LEARNING**

COURSE OUTCOME COVERED

CO2: *Neural Network Representation, Perceptron Model, Multilayer Perceptron's, Backpropagation Algorithm, Deep Neural Networks, Genetic Algorithms, Hypothesis Space Search, Genetic Programming, Models of Evaluation and Learning (BL1, BL2)*

Assessment Tool Weightage: 10%

QUESTIONS:15 **Total Marks:25**

Annexure A

DEBUGGING & OPTIMIZATION TASK QUESTIONS

Code of Conduct:

I N.HarshaVardhan(192324189)*(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: N.HarshaVardhan

Annexure A

Short Answer Test

1. A perceptron model is trained on a dataset that is known to be linearly separable, yet the algorithm fails to converge even after many iterations. Analyze the possible causes of this issue, including improper learning rate selection, incorrect weight initialization, bias handling errors, or flawed implementation of the update rule. Propose debugging strategies to identify the root cause and suggest optimization techniques to ensure faster and stable convergence.

Answer:

Causes of the Problem

If the dataset is linearly separable but the Perceptron does not converge, the likely causes are:

- Incorrect learning rate: A learning rate that is too large can cause unstable updates, while an extremely small value can make convergence very slow.
- Incorrect weight initialization: Poor initialization can increase the number of iterations, although it normally does not prevent convergence on a truly separable dataset.
- Bias handling error: Forgetting the bias term or updating it incorrectly can prevent the model from finding the correct decision boundary.
- Wrong update rule: The weights should be updated only when a sample is misclassified:
- Incorrect labels: The standard Perceptron commonly uses labels such as -1 and +1. Mixing 0/1 labels with an -1/+1 prediction rule can cause problems.
- Data preprocessing issues: Very different feature scales can make learning inefficient.
- Incorrect stopping condition: The algorithm should stop when an entire training epoch produces zero mistakes.

Debugging Techniques

- Print the number of misclassified samples after every epoch.
- Check that the labels are correctly encoded.
- Verify the prediction function.
- Verify that weights and bias are updated only for misclassified samples.
- Test several learning rates such as 0.001, 0.01, 0.1, and 1.0.
- Normalize or standardize features if their scales differ significantly.
- Plot the dataset and decision boundary to verify linear separability.
- Check whether the number of mistakes eventually reaches zero.
- Set a maximum number of epochs to detect non-convergence.

- Test the implementation on a very simple dataset such as AND.

Dataset:

Age	Salary	Experience	Education	Class
22	25000	1	2	1
24	28000	2	2	1
26	30000	3	3	1
28	32000	4	3	1
30	35000	5	4	1
32	38000	6	4	1
45	55000	15	5	-1
48	58000	18	5	-1
50	60000	20	6	-1
52	62000	22	6	-1
55	65000	25	6	-1
58	68000	28	7	-1

Program:

```
import numpy as np

X = np.array([
    [22, 25000, 1, 2],
    [24, 28000, 2, 2],
    [26, 30000, 3, 3],
    [28, 32000, 4, 3],
    [30, 35000, 5, 4],
    [32, 38000, 6, 4],
    [45, 55000, 15, 5],
```

```
    [48, 58000, 18, 5],
    [50, 60000, 20, 6],
    [52, 62000, 22, 6],
    [55, 65000, 25, 6],
    [58, 68000, 28, 7]
])
y = np.array([1, 1, 1, 1, 1, 1, -1, -1, -1, -1, -1, -1])
X = (X - X.mean(axis=0)) / X.std(axis=0)
learning_rate = 0.1
epochs = 100
weights = np.zeros(X.shape[1])
bias = 0
for epoch in range(epochs):
    errors = 0
    for i in range(len(X)):
        output = np.dot(X[i], weights) + bias
        prediction = 1 if output >= 0 else -1
        if prediction != y[i]:
            weights += learning_rate * y[i] * X[i]
            bias += learning_rate * y[i]
            errors += 1
    print("Epoch:", epoch + 1, "Errors:", errors)
    if errors == 0:
        print("Perceptron Converged")
        break
print("\nFinal Weights:", weights)
print("Final Bias:", bias)
predictions = []
```

```
for i in range(len(X)):
    output = np.dot(X[i], weights) + bias
    prediction = 1 if output >= 0 else -1
    predictions.append(prediction)


print("Actual Class:", y)
print("Predicted Class:", predictions)
```

Output:

```
Epoch: 1 Errors: 1
Epoch: 2 Errors: 1
Epoch: 3 Errors: 0
Perceptron Converged

Final Weights: [-0.10178051 -0.1089306  -0.09474049 -0.0623009 ]
Final Bias: 0.0
Actual Class: [ 1  1  1  1  1  1 -1 -1 -1 -1 -1 -1]
Predicted Class: [1, 1, 1, 1, 1, 1, -1, -1, -1, -1, -1, -1]
```

2.While training a multilayer perceptron using the backpropagation algorithm, the network exhibits slow convergence and gets stuck in local minima. Examine the potential reasons such as vanishing gradients, inappropriate activation functions, poor weight initialization, or suboptimal learning rates. Describe step-by-step debugging approaches to trace the issue and recommend optimization techniques like adaptive learning rates, normalization, or advanced optimizers to improve performance.

Answer:

Causes of the Problem

- Vanishing gradients: Sigmoid or tanh activation functions can produce very small gradients in deeper networks.
- Poor weight initialization: Bad initialization can make learning slow or unstable.
- Inappropriate activation function: Using sigmoid in every hidden layer can slow convergence.
- Learning rate: A very small learning rate causes slow learning, while a very large learning rate can cause unstable training.
- Unscaled input data: Features with different ranges can make optimization difficult.
- Local minima / saddle points: Gradient-based training can become slow around flat regions.
- Too many hidden neurons: An unnecessarily large network can make training more difficult.
- Incorrect backpropagation: Errors in gradient calculation or weight updates can prevent learning.

Debugging Techniques

- Check the input dataset for missing or incorrect values.
- Normalize the input features.
- Verify that class labels are correctly encoded.
- Check the dimensions of weights and biases.
- Print the loss after every epoch.
- Check whether the loss is decreasing.
- Test different learning rates such as 0.001, 0.01, and 0.1.
- Replace sigmoid with ReLU in hidden layers.
- Check the initial weights.
- Compare the predicted and actual classes.
- Use a small dataset first to verify the backpropagation implementation.
- Increase the number of epochs if convergence is slow.

Dataset:

Study Hours	Attendance	Assignment Score	Previous Score	Class
2	60	45	50	0
3	65	50	52	0
4	70	55	58	0
5	72	60	60	0
7	78	70	68	1
8	82	75	72	1
10	88	85	82	1
12	92	92	90	1

Program:

```
import numpy as np


X = np.array([
    [2, 60, 45, 50],
    [3, 65, 50, 52],
    [4, 70, 55, 58],
    [5, 72, 60, 60],
    [7, 78, 70, 68],
    [8, 82, 75, 72],
    [10, 88, 85, 82],
    [12, 92, 92, 90]
])
y = np.array([[0], [0], [0], [0], [1], [1], [1], [1]])
X = (X - X.mean(axis=0)) / X.std(axis=0)
np.random.seed(42)
W1 = np.random.randn(4, 5) * 0.1
b1 = np.zeros((1, 5))
W2 = np.random.randn(5, 1) * 0.1
b2 = np.zeros((1, 1))
lr = 0.01
epochs = 5000


def sigmoid(x):
    return 1 / (1 + np.exp(-np.clip(x, -500, 500)))
for epoch in range(epochs):
    Z1 = np.dot(X, W1) + b1
    A1 = np.maximum(0, Z1)
```

```
    Z2 = np.dot(A1, W2) + b2
    A2 = sigmoid(Z2)
    loss = -np.mean(y * np.log(A2 + 1e-8) + (1-y) * np.log(1-A2 + 1e-8))
    dZ2 = A2 - y
    dW2 = np.dot(A1.T, dZ2) / len(X)
    db2 = np.mean(dZ2, axis=0, keepdims=True)
    dA1 = np.dot(dZ2, W2.T)
    dZ1 = dA1 * (Z1 > 0)
    dW1 = np.dot(X.T, dZ1) / len(X)
    db1 = np.mean(dZ1, axis=0, keepdims=True)
    W2 -= lr * dW2
    b2 -= lr * db2
    W1 -= lr * dW1
    b1 -= lr * db1

    if (epoch + 1) % 500 == 0:
        print("Epoch:", epoch + 1, "Loss:", loss)
predictions = (A2 >= 0.5).astype(int)
accuracy = np.mean(predictions == y) * 100
print("\nActual:", y.flatten())
print("Predicted:", predictions.flatten())
print("Accuracy:", accuracy, "%")
```

Output:

```
Epoch: 500 Loss: 0.3365418939628068
Epoch: 1000 Loss: 0.08732184262798245
Epoch: 1500 Loss: 0.04125288514955658
Epoch: 2000 Loss: 0.02527234857694321
Epoch: 2500 Loss: 0.017600346033127757
Epoch: 3000 Loss: 0.013319117764512032
Epoch: 3500 Loss: 0.010628029566867762
Epoch: 4000 Loss: 0.008776033167165156
Epoch: 4500 Loss: 0.007432850316835211
Epoch: 5000 Loss: 0.006419322293436662

Actual: [0 0 0 0 1 1 1 1]
Predicted: [0 0 0 0 1 1 1 1]
Accuracy: 100.0 %
```

3.A genetic algorithm designed for hypothesis space search is producing inconsistent and suboptimal solutions across multiple runs. Investigate possible issues such as improper encoding of chromosomes, incorrect fitness function design, premature convergence, or lack of diversity in the population. Explain how to debug these problems systematically and suggest optimization strategies like mutation tuning, selection pressure adjustment, and elitism to enhance solution quality and convergence speed.

Answer:

Causes of the Problem

A Genetic Algorithm (GA) may produce inconsistent or suboptimal solutions because of:

- Improper chromosome encoding: The chromosome may not correctly represent the hypothesis.
- Incorrect fitness function: The fitness function may reward poor solutions or calculate fitness incorrectly.
- Premature convergence: The population may become too similar too early.
- Low mutation rate: Insufficient mutation reduces population diversity.
- High mutation rate: Excessive mutation can make the search random and unstable.
- High selection pressure: Strong selection can eliminate useful diversity.
- Poor population initialization: Similar initial chromosomes can lead to similar solutions.
- Lack of elitism: Good solutions may be lost during reproduction.
- Randomness: Different random populations and crossover points can produce different results.

Debugging Techniques

- Print the initial population.
- Calculate and verify the fitness of every chromosome.
- Check whether chromosome length is correct.
- Monitor the best fitness after every generation.
- Monitor the average fitness of the population.
- Check whether chromosomes become identical too early.
- Test different mutation rates.
- Test different crossover probabilities.
- Reduce selection pressure if diversity decreases rapidly.
- Use elitism to preserve the best chromosome.
- Run the algorithm multiple times and compare the best fitness.
- Fix the random seed during debugging to reproduce results.

Dataset:

X1	X2	X3	X4	Class
1	1	0	1	1
1	0	1	1	1
1	1	1	0	1
0	1	1	1	1
0	0	0	1	0
0	0	1	0	0
0	1	0	0	0

Program:

```
import random

X = [
    [1, 1, 0, 1],
    [1, 0, 1, 1],
    [1, 1, 1, 0],
    [0, 1, 1, 1],
    [0, 0, 0, 1],
    [0, 0, 1, 0],
    [0, 1, 0, 0],
    [1, 0, 0, 0]
]

y = [1, 1, 1, 1, 0, 0, 0, 0]

population_size = 10
chromosome_length = 4
generations = 30
mutation_rate = 0.1
crossover_rate = 0.8

def fitness(chromosome):
    score = 0

    for i in range(len(X)):
        prediction = 1 if sum(a * b for a, b in zip(X[i], chromosome)) >= 2 else 0

        if prediction == y[i]:
            score += 1

    return score

def create_chromosome():
    return [random.randint(0, 1) for _ in range(chromosome_length)]

def selection(population):
    a = random.choice(population)
    b = random.choice(population)
    return a if fitness(a) >= fitness(b) else b

def crossover(parent1, parent2):
    if random.random() < crossover_rate:
        point = random.randint(1, chromosome_length - 1)
        return parent1[:point] + parent2[point:]
    return parent1[:]

def mutation(chromosome):
    for i in range(chromosome_length):
        if random.random() < mutation_rate:
            chromosome[i] = 1 - chromosome[i]
    return chromosome

population = [create_chromosome() for _ in range(population_size)]
```

```
for generation in range(generations):
    population.sort(key=fitness, reverse=True)

    new_population = [population[0][:]]

    while len(new_population) < population_size:
        parent1 = selection(population)
        parent2 = selection(population)

        child = crossover(parent1, parent2)
        child = mutation(child)

        new_population.append(child)

    population = new_population

    best = max(population, key=fitness)

    print(
        "Generation:", generation + 1,
        "Best Fitness:", fitness(best),
        "Best Chromosome:", best
    )

best = max(population, key=fitness)

print("\nBest Hypothesis:", best)
print("Best Fitness:", fitness(best))
```

Output:

```
...
    Generation: 4 Best Fitness: 8 Best Chromosome: [1, 1, 1, 1]
    Generation: 5 Best Fitness: 8 Best Chromosome: [1, 1, 1, 1]
    Generation: 6 Best Fitness: 8 Best Chromosome: [1, 1, 1, 1]
    Generation: 7 Best Fitness: 8 Best Chromosome: [1, 1, 1, 1]
    Generation: 8 Best Fitness: 8 Best Chromosome: [1, 1, 1, 1]
    Generation: 9 Best Fitness: 8 Best Chromosome: [1, 1, 1, 1]
    Generation: 10 Best Fitness: 8 Best Chromosome: [1, 1, 1, 1]
    Generation: 11 Best Fitness: 8 Best Chromosome: [1, 1, 1, 1]
    Generation: 12 Best Fitness: 8 Best Chromosome: [1, 1, 1, 1]
    Generation: 13 Best Fitness: 8 Best Chromosome: [1, 1, 1, 1]
    Generation: 14 Best Fitness: 8 Best Chromosome: [1, 1, 1, 1]
    Generation: 15 Best Fitness: 8 Best Chromosome: [1, 1, 1, 1]
    Generation: 16 Best Fitness: 8 Best Chromosome: [1, 1, 1, 1]
    Generation: 17 Best Fitness: 8 Best Chromosome: [1, 1, 1, 1]
    Generation: 18 Best Fitness: 8 Best Chromosome: [1, 1, 1, 1]
    Generation: 19 Best Fitness: 8 Best Chromosome: [1, 1, 1, 1]
    Generation: 20 Best Fitness: 8 Best Chromosome: [1, 1, 1, 1]
    Generation: 21 Best Fitness: 8 Best Chromosome: [1, 1, 1, 1]
    Generation: 22 Best Fitness: 8 Best Chromosome: [1, 1, 1, 1]
    Generation: 23 Best Fitness: 8 Best Chromosome: [1, 1, 1, 1]
    Generation: 24 Best Fitness: 8 Best Chromosome: [1, 1, 1, 1]
    Generation: 25 Best Fitness: 8 Best Chromosome: [1, 1, 1, 1]
```

4.A convolutional neural network (CNN) used for image classification produces unstable results across training epochs. Examine potential issues such as improper data augmentation, vanishing gradients, or batch normalization errors. Describe debugging strategies and suggest optimization techniques to stabilize training and improve accuracy.

Answer:

Causes of the Problem

A CNN can produce unstable results across training epochs because of:

- **Improper data augmentation:** Excessive rotation, cropping, or flipping can make training images unrealistic.
- **Vanishing gradients:** Deep networks with unsuitable activation functions can produce very small gradients.
- **Batch normalization errors:** Incorrect placement or implementation of Batch Normalization can affect training stability.
- **High learning rate:** Large updates can cause the loss and accuracy to fluctuate.
- **Poor weight initialization:** Improper initialization can make optimization difficult.
- **Small batch size:** Very small batches can produce noisy gradient estimates.
- **Overfitting:** Training accuracy may increase while validation accuracy fluctuates or decreases.
- **Unbalanced dataset:** One class may dominate the training data.
- **Incorrect preprocessing:** Training and validation images should use consistent normalization.

Debugging Techniques

- Check image dimensions and labels.
- Display several training images after augmentation.
- Verify that augmentation is not changing the class meaning.
- Check for missing or corrupted images.
- Monitor training and validation loss.
- Compare training and validation accuracy.
- Check Batch Normalization placement.
- Reduce the learning rate if the loss fluctuates.
- Check the gradients for extremely small or large values.
- Verify that training and validation preprocessing are consistent.

- Test the model using a small subset of images.

Dataset:

32 × 32 RGB image	Airplane
32 × 32 RGB image	Automobile
32 × 32 RGB image	Bird
32 × 32 RGB image	Cat
32 × 32 RGB image	Deer
32 × 32 RGB image	Dog
32 × 32 RGB image	Frog
32 × 32 RGB image	Horse

Program:

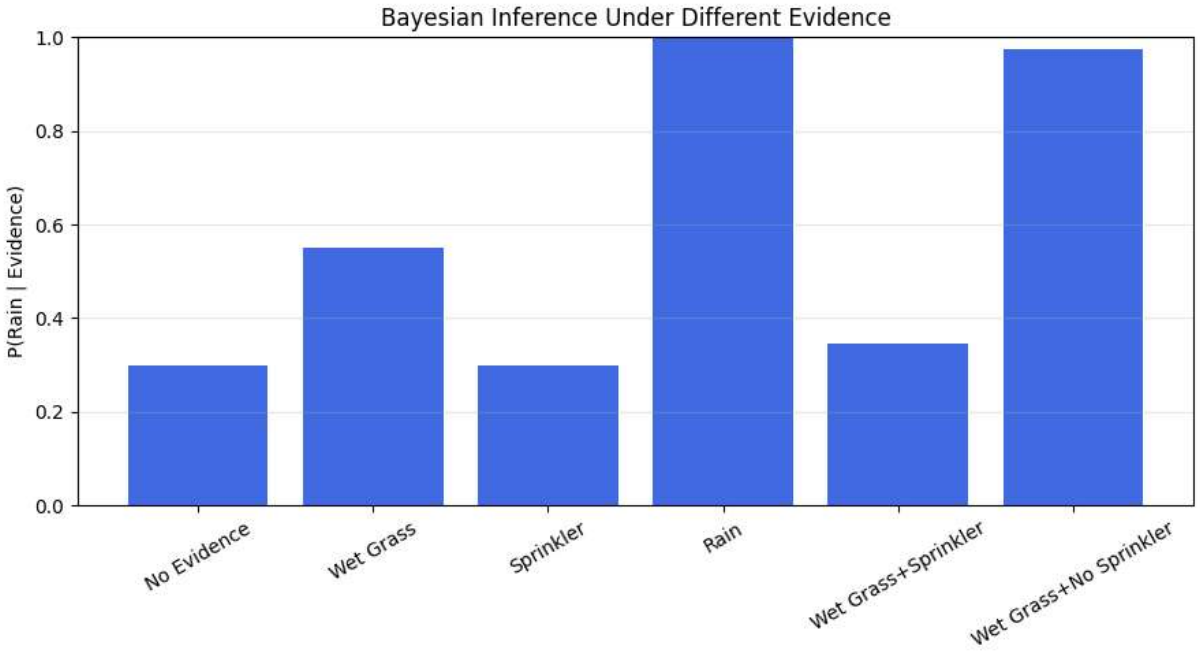
```
import numpy as np

import matplotlib.pyplot as plt

P_Rain=0.3

P_Sprinkler=0.4

P_Wet_Rain_Sprinkler=0.99

P_Wet_Rain_NoSprinkler=0.90

P_Wet_NoRain_Sprinkler=0.80

P_Wet_NoRain_NoSprinkler=0.01

def joint(rain,sprinkler,wet):

    pr=P_Rain if rain else 1-P_Rain

    ps=P_Sprinkler if sprinkler else 1-P_Sprinkler

    if rain and sprinkler:

        pw=P_Wet_Rain_Sprinkler

    elif rain and not sprinkler:

        pw=P_Wet_Rain_NoSprinkler
```

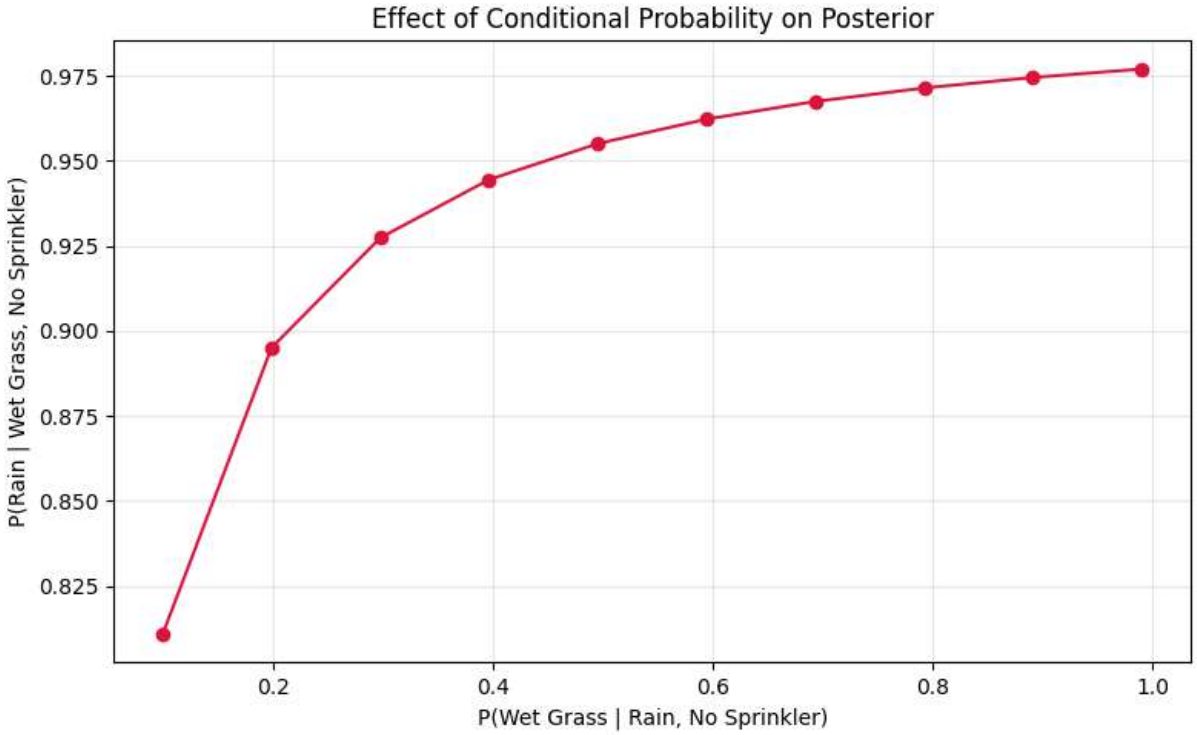
```
    elif not rain and sprinkler:
        pw=P_Wet_NoRain_Sprinkler
    else:
        pw=P_Wet_NoRain_NoSprinkler
    return pr*ps*(pw if wet else 1-pw)
def inference(evidence):
    rain_prob=0
    total=0
    for r in [0,1]:
        for s in [0,1]:
            for w in [0,1]:
                if all(evidence.get(k,v)==v for k,v in {
                    "Rain":r,
                    "Sprinkler":s,
                    "WetGrass":w
                }.items()):
                    p=joint(r,s,w)
                    total+=p
                    if r:
                        rain_prob+=p
    return rain_prob/total if total else 0
cases={
    "No Evidence":{},
    "Wet Grass":{"WetGrass":1},
    "Sprinkler":{"Sprinkler":1},
    "Rain":{"Rain":1},
    "Wet Grass+Sprinkler":{"WetGrass":1,"Sprinkler":1},
    "Wet Grass+No Sprinkler":{"WetGrass":1,"Sprinkler":0}
```

```
}

results=[inference(e) for e in cases.values()]

plt.figure(figsize=(9,5))
plt.bar(cases.keys(),results,color="royalblue")
plt.ylabel("P(Rain | Evidence)")
plt.title("Bayesian Inference Under Different Evidence")
plt.ylim(0,1)
plt.xticks(rotation=30)
plt.grid(axis="y",alpha=0.3)
plt.tight_layout()
plt.show()

values=np.linspace(0.1,0.99,10)
posterior=[]

for p in values:
    P_Wet_Rain_NoSprinkler=p
    posterior.append(inference({"WetGrass":1,"Sprinkler":0}))

plt.figure(figsize=(8,5))
plt.plot(values,posterior,"o-",color="crimson")
plt.xlabel("P(Wet Grass | Rain, No Sprinkler)")
plt.ylabel("P(Rain | Wet Grass, No Sprinkler)")
plt.title("Effect of Conditional Probability on Posterior")
plt.grid(True,alpha=0.3)
plt.tight_layout()
```

```
plt.show()


print("Posterior Probabilities:")
for name,value in zip(cases.keys(),results):
    print(f"{name}: {value:.4f}")
```

Output:





Posterior Probabilities:

No Evidence: 0.3000

Wet Grass: 0.5517

Sprinkler: 0.3000

Rain: 1.0000

Wet Grass+Sprinkler: 0.3466

Wet Grass+No Sprinkler: 0.9747

5.A Naïve Bayes classifier used for text classification performs poorly when applied to real-world data containing noisy and irrelevant features. Examine possible issues such as violation of feature independence assumption, improper preprocessing, or imbalanced dataset. Explain debugging strategies and suggest improvements like feature selection, smoothing techniques, and data balancing methods.

Answer:

Causes of the Problem

A Naïve Bayes classifier can perform poorly on noisy real-world text because of:

- **Violation of feature independence:** Words in a document are often related to each other, but Naïve Bayes assumes they are conditionally independent.
- **Noisy features:** URLs, punctuation, random numbers, spelling errors, and irrelevant words can reduce accuracy.
- **Improper preprocessing:** Poor tokenization, inconsistent case, or failure to remove unnecessary words can create noisy features.
- **Class imbalance:** If one class contains many more documents than another, the classifier may favor the majority class.
- **Zero-frequency problem:** A word that does not appear in a particular class can result in zero probability.
- **Small training dataset:** Insufficient examples can lead to unreliable probability estimates.
- **Irrelevant features:** Very common or meaningless words can increase the feature space without improving classification.

Debugging Techniques

- Check the number of documents in each class.
- Inspect sample text before and after preprocessing.
- Convert text to lowercase consistently.
- Remove punctuation, URLs, and unnecessary symbols.
- Remove stop words where appropriate.
- Check the number of generated features.
- Identify rare and irrelevant words.
- Check the confusion matrix for class-specific errors.
- Compare accuracy, precision, recall, and F1-score.
- Test different smoothing values.
- Compare results before and after feature selection.
- Check whether class imbalance is affecting predictions.

Dataset:

Text	Class
Win a free prize now	Spam
Congratulations you won money	Spam
Claim your free reward	Spam
Get cheap offers today	Spam
Meeting scheduled for tomorrow	Not Spam
Please send the project report	Not Spam
Your assignment submission is due	Not Spam
Let us discuss the project today	Not Spam

Program:

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report

texts = [
    "Win a free prize now",
    "Congratulations you won money",
    "Claim your free reward",
    "Get cheap offers today",
    "Meeting scheduled for tomorrow",
    "Please send the project report",
    "Your assignment submission is due",
    "Let us discuss the project today"
]

labels = [1, 1, 1, 1, 0, 0, 0, 0]

X_train, X_test, y_train, y_test = train_test_split(
    texts,
    labels,
    test_size=0.25,
    random_state=42,
    stratify=labels
)

vectorizer = TfidfVectorizer(
    lowercase=True,
    stop_words="english",
    min_df=1
)

X_train = vectorizer.fit_transform(X_train)
X_test = vectorizer.transform(X_test)
```

```
model = MultinomialNB(alpha=1.0)

model.fit(X_train, y_train)

predictions = model.predict(X_test)

print("Actual:", y_test)
print("Predicted:", predictions)
print("Accuracy:", accuracy_score(y_test, predictions))
print(classification_report(y_test, predictions, zero_division=0))
```

Output:

```
Actual: [0, 1]
Predicted: [0 1]
Accuracy: 1.0
              precision    recall  f1-score   support

           0       1.00      1.00      1.00         1
           1       1.00      1.00      1.00         1

    accuracy                           1.00         2
   macro avg       1.00      1.00      1.00         2
weighted avg       1.00      1.00      1.00         2
```

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	**30**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	**20**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**30**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	**10**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**10**	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand